The problem

Consider a model that predicts next-month excess returns $r_{i,t+1}$ from a vector of firm characteristics $x_{i,t} \in \mathbb{R}^P$ (size, book-to-market, momentum, volatility, ...). The dataset is a panel of N stocks over T months. The model has one or more *hyperparameters* —e.g. the ℓ_1 penalty λ in LASSO, or (number of trees, max. depth, min. leaf size) in a random forest. These cannot be learned alongside the coefficients: doing so would always pick the value that minimises training error (e.g. $\lambda = 0$), which overfits.

The three-way, chronologically ordered split

Because returns are a time series, observations **must not be shuffled**—random splitting would leak future information into the training set (look-ahead bias). The sample is partitioned into three *disjoint, time-ordered* sub-periods:

1. **Training (earliest).** Estimates parameters $\hat{\beta}$ conditional on fixed hyperparameter values.
2. **Validation (middle).** Tunes hyperparameters. The training fit is used to forecast the validation sample; an objective function (MSE, or a robust Huber loss for outlier protection) is evaluated on those forecasts.
3. **Testing (latest).** Used for neither estimation nor tuning. Gives a *truly* out-of-sample estimate of predictive performance.

Typical proportions are roughly 50 / 25 / 25%. Gu, Kelly, and Xiu (2020), for example, train on 1957–1974, validate on 1975–1986, and test on 1987 onward.

The iterative tuning procedure

For each candidate hyperparameter λ_k on a grid:

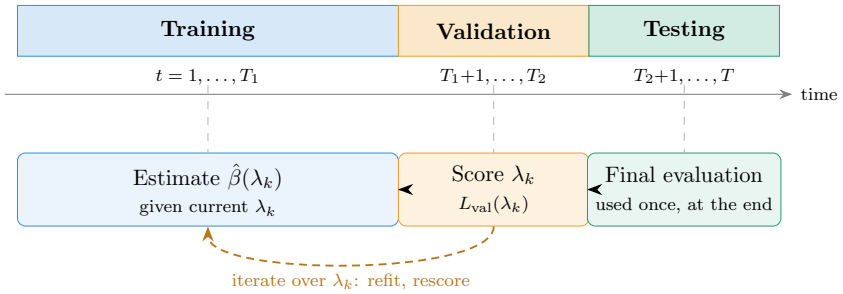
1. Re-estimate on the training sample, $\hat{\beta}(\lambda_k) = \arg\min_{\beta} \{ L(\beta; \mathcal{D}_{\text{train}}) + \lambda_k P(\beta) \}$, where L is the training loss and $P(\beta)$ a complexity penalty (e.g. $\sum_j |\beta_j|$ for LASSO, $\sum_j \beta_j^2$ for ridge).
2. Forecast the validation sample with $\hat{\beta}(\lambda_k)$ and record $L_{\text{val}}(\lambda_k)$.

Select $\lambda^\star = \arg\min_k L_{\text{val}}(\lambda_k)$, and report performance on the testing sample using $\hat{\beta}(\lambda^\star)$. Since validation fits feed back into estimation via the choice of $\lambda^\star$, they are *not* truly out-of-sample; only the testing period is.

Typical hyperparameters by model

Model	Tuned on validation
LASSO / Ridge / Elastic net	penalty λ (and mix α)
Principal-component regression	number of components K
Random forest / GBRT	max depth, # trees, min leaf size, learning rate
Neural networks	depth, width, L_1/L_2 penalties, dropout, learning rate

Scheme



Further reading. Details of the sample-splitting scheme are given in **Appendix D**; a summary of the hyperparameter tuning schemes for each individual model is provided in **Appendix E**.